



CS240 Algorithm Design and Analysis

Lecture 4

Divide and Conquer (Cont.)

Qingying Hao
Spring 2026
2026.03.12



Last Time – What you need to know



- **Basic idea**
 - Make the locally optimal choice at each step
- **Algorithms**
 - Optimal Caching
 - Evict item that is requested farthest in future
- **Proof skills**
 - **Greedy algorithm stays ahead.** Show that after each step of the greedy algorithm, its solution is at least as good as any other algorithm's
 - **Exchange argument.** Gradually transform any solution to the one found by the greedy algorithm without hurting its quality
- **Mergesort**
- **Closest Pair of Points: Analysis**





Integer Multiplication (Revisit)





Divide-and-Conquer Multiplication: Warmup

- To multiply two n -digit integers:
 - Multiply four $\frac{1}{2}n$ - digit integers
 - Add two $\frac{1}{2}n$ -digit integers, and shift to obtain result

$$x = \underbrace{1000}_{x_1} \underbrace{1101}_{x_0}$$

$$\begin{aligned}x &= 2^{n/2} \cdot x_1 + x_0 \\y &= 2^{n/2} \cdot y_1 + y_0 \\xy &= (2^{n/2} \cdot x_1 + x_0)(2^{n/2} \cdot y_1 + y_0) = 2^n \cdot x_1 y_1 + 2^{n/2} \cdot (x_1 y_0 + x_0 y_1) + x_0 y_0\end{aligned}$$

$$T(n) = \underbrace{4T(n/2)}_{\text{recursive calls}} + \underbrace{\Theta(n)}_{\text{add, shift}} \Rightarrow T(n) = \Theta(n^2)$$

↑
assumes n is a power of 2

level 0: 1 problems
level 1: 4 problems
level 2: 16 problems
level 3: 64 problems
level k : 4^k
 $n/2^k=1, k=\log_2 n$

$$4^{\log_2 n} = n^{\log_2 4} = n^2$$





Karatsuba Multiplication



- To multiply two n -digit integers:
 - Add two $\frac{1}{2}n$ digit integers
 - Multiply **three** $\frac{1}{2}n$ -digit integers
 - Add, subtract, and shift $\frac{1}{2}n$ -digit integers to obtain results

$$\begin{aligned}x &= 2^{n/2} \cdot x_1 + x_0 \\y &= 2^{n/2} \cdot y_1 + y_0 \\xy &= 2^n \cdot x_1 y_1 + 2^{n/2} \cdot (x_1 y_0 + x_0 y_1) + x_0 y_0 \\&= 2^n \cdot x_1 y_1 + 2^{n/2} \cdot ((x_1 + x_0)(y_1 + y_0) - x_1 y_1 - x_0 y_0) + x_0 y_0\end{aligned}$$

A B A C C

- **Theorem.** Can multiply two n -digit integers in $O(n^{1.585})$ bit operations

$$\begin{aligned}T(n) &\leq \underbrace{T(\lfloor n/2 \rfloor) + T(\lceil n/2 \rceil) + T(1 + \lceil n/2 \rceil)}_{\text{recursive calls}} + \underbrace{\Theta(n)}_{\text{add, subtract, shift}} \\&\Rightarrow T(n) = O(n^{\log_2 3}) = O(n^{1.585})\end{aligned}$$



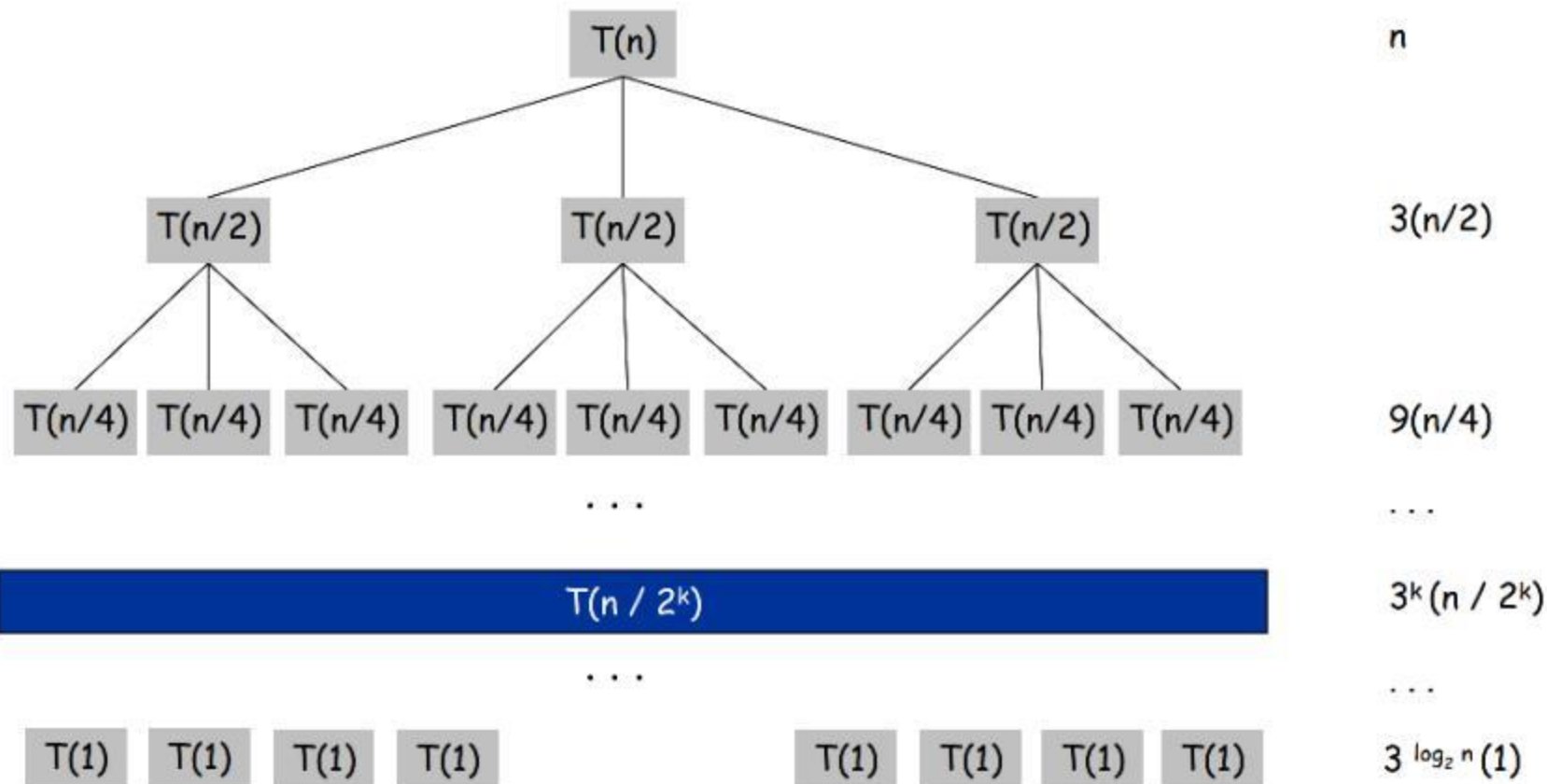


Karatsuba: Recursion Tree



$$T(n) = \begin{cases} 1 & \text{if } n = 1 \\ 3T(n/2) + n & \text{otherwise} \end{cases}$$

$$T(n) = \sum_{k=0}^{\log_2 n} n \left(\frac{3}{2}\right)^k = \frac{\left(\frac{3}{2}\right)^{1+\log_2 n} - 1}{\frac{3}{2} - 1} n = 3n^{\log_2 3} - 2n$$





Matrix Multiplication





Matrix Multiplication

- **Matrix multiplication.** Given two n-by-n matrices A and B, compute $C = AB$

$$c_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

$$\begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1n} \\ b_{21} & b_{22} & \cdots & b_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{nn} \end{bmatrix}$$

- **Brute force.** $\Theta(n^3)$ arithmetic operations.
- **Fundamental question.** Can we improve upon brute force?

```
for i = 1..n
  for j = 1..n
    for k = 1..n
      C[i][j] += A[i][k] * B[k][j]
```





Matrix Multiplication: Warmup

- **Divide-and-conquer.**

- Divide: partition A and B into $\frac{1}{2}n$ -by- $\frac{1}{2}n$ blocks
- Conquer: multiply 8 $\frac{1}{2}n$ -by- $\frac{1}{2}n$ recursively
- Combine: add appropriate products using 4 matrix additions

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$\begin{aligned} C_{11} &= (A_{11} \times B_{11}) + (A_{12} \times B_{21}) \\ C_{12} &= (A_{11} \times B_{12}) + (A_{12} \times B_{22}) \\ C_{21} &= (A_{21} \times B_{11}) + (A_{22} \times B_{21}) \\ C_{22} &= (A_{21} \times B_{12}) + (A_{22} \times B_{22}) \end{aligned}$$

Layer 0: 1 problem size n

Layer 1 : 8 problems size $n/2$

Layer 2 : 8^2 problems size $n/4$

Layer k :

8^k problems size $n / 2^k$

Recursion stops at
 $n / 2^k = 1, k = \log_2 n$

So:

$$8^{\log_2 n} = n^{\log_2 8} = n^3$$

$$T(n) = \underbrace{8T(n/2)}_{\text{recursive calls}} + \underbrace{\Theta(n^2)}_{\text{add, form submatrices}} \Rightarrow T(n) = \Theta(n^3)$$





Matrix Multiplication: Warmup

- **Divide-and-conquer.**

- Divide: partition A and B into $\frac{1}{2}n$ -by- $\frac{1}{2}n$ blocks
- Conquer: multiply 8 $\frac{1}{2}n$ -by- $\frac{1}{2}n$ recursively
- Combine: add appropriate products using 4 matrix additions

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$\begin{aligned} C_{11} &= (A_{11} \times B_{11}) + (A_{12} \times B_{21}) \\ C_{12} &= (A_{11} \times B_{12}) + (A_{12} \times B_{22}) \\ C_{21} &= (A_{21} \times B_{11}) + (A_{22} \times B_{21}) \\ C_{22} &= (A_{21} \times B_{12}) + (A_{22} \times B_{22}) \end{aligned}$$

$$T(n) = \underbrace{8T(n/2)}_{\text{recursive calls}} + \underbrace{\Theta(n^2)}_{\text{add, form submatrices}} \Rightarrow T(n) = \Theta(n^3)$$

Layer 0: 1 problem size n cost = n^2

Layer 1: 8 problems size $n/2$, $(n/2)^2$
 $8 \times (n/2)^2 = 8 \times n^2/4 = 2n^2$

Layer 2: 8^2 problems size $n/4$, $(n/4)^2$
 $8^2 \times (n/4)^2 = 64 \times n^2/16 = 4n^2$

Layer k : 8^k problems size $n/2^k$
 $8^k \times (n/2^k)^2 = 8^k \times n^2/4^k$
 $= n^2 \times (8/4)^k$
 $= n^2 \times 2^k$

$k = \log_2 n$

Total cost: $n^2 \times 2^{\log_2 n} = n^3$





Matrix Multiplication: Key Idea



- **Key idea.** Multiply 2-by-2 block matrices with only **7** multiplications

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \times \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$C_{11} = P_5 + P_4 - P_2 + P_6$$

$$C_{12} = P_1 + P_2$$

$$C_{21} = P_3 + P_4$$

$$C_{22} = P_5 + P_1 - P_3 - P_7$$

$$P_1 = A_{11} \times (B_{12} - B_{22})$$

$$P_2 = (A_{11} + A_{12}) \times B_{22}$$

$$P_3 = (A_{21} + A_{22}) \times B_{11}$$

$$P_4 = A_{22} \times (B_{21} - B_{11})$$

$$P_5 = (A_{11} + A_{22}) \times (B_{11} + B_{22})$$

$$P_6 = (A_{12} - A_{22}) \times (B_{21} + B_{22})$$

$$P_7 = (A_{11} - A_{21}) \times (B_{11} + B_{12})$$

- 7 multiplications.
- 18 = 10 + 8 additions (or subtractions)





Fast Matrix Multiplication



- **Fast matrix multiplication (Strassen, 1969)**
 - Divide: partition A and B into $\frac{1}{2}n$ -by- $\frac{1}{2}n$ blocks
 - Compute: 14 $\frac{1}{2}n$ -by- $\frac{1}{2}n$ matrices via 10 matrix additions
 - Conquer: multiply 7 $\frac{1}{2}n$ -by- $\frac{1}{2}n$ matrices recursively
 - Combine: 7 products into 4 terms using 8 matrix additions
- **Analysis**
 - Assume n is a power of 2
 - $T(n)$ = # arithmetic operations

$$T(n) = \underbrace{7T(n/2)}_{\text{recursive calls}} + \underbrace{\Theta(n^2)}_{\text{add, subtract}} \Rightarrow T(n) = \Theta(n^{\log_2 7}) = O(n^{2.81})$$

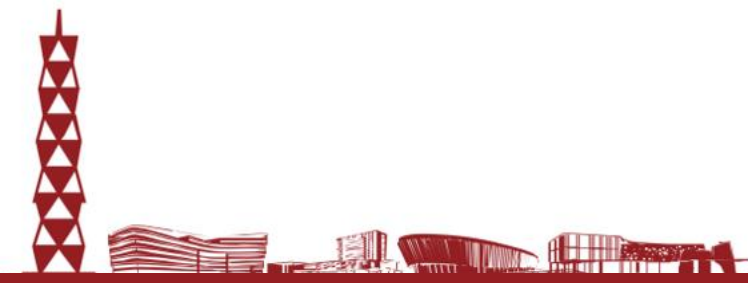




Fast Matrix Multiplication in Practice



- **Common misperception:** “Strassen is only a theoretical curiosity.”
 - Advanced computation group at apple computer reports 8* speedup on G4 Velocity Engine when $n \sim 2,500$
 - Range of instances where it’s useful is a subject of controversy
- **Remark.** Can “Strassenize” $Ax = b$, determinant, eigenvalues, and other matrix ops





Fast Matrix Multiplication in Theory



- **Q.** Multiply two 2-by-2 matrices with only 7 scalar multiplications?

- **A.** Yes! [Strassen, 1969]

$$\Theta(n^{\log_2 7}) = O(n^{2.81})$$

- **Q.** Multiply two 2-by-2 matrices with only 6 scalar multiplications?

- **A.** Impossible. [Hopcroft and Kerr, 1971]

$$\Theta(n^{\log_2 6}) = O(n^{2.59})$$

- **Q.** Two 3-by-3 matrices with only 21 scalar multiplications?

- **A.** Also impossible.

$$\Theta(n^{\log_3 21}) = O(n^{2.77})$$

- **Q.** Two 70-by-70 matrices with only 143,640 scalar multiplications?

- **A.** Yes!

$$\Theta(n^{\log_{70} 143640}) = O(n^{2.80})$$





Fast Matrix Multiplication in Theory



- **Decimal wars**
 - December 1979: $O(n^{2.521813})$
 - January 1980: $O(n^{2.521801})$
 - ...
 - 1987: $O(n^{2.375477})$
 - 2010: $O(n^{2.374})$
 - 2011: $O(n^{2.3728642})$
 - 2014: $O(n^{2.3728639})$
- **Best known.** $O(n^{2.3728639})$ [Francois Gall, 2014]
- **Conjecture.** $O(2+e)$ for any $e > 0$
- **Caveat.** Theoretical improvements to Strassen are progressively less practical





Convolution and FFT





Fast Fourier Transform: Applications



- **Applications**

- Optics, acoustics, quantum physics, telecommunications, control systems, signal processing, speech recognition, data compression, image processing
- DVD, JPEG, MP3, MRI, CAT scan
- Numerical solutions to Poisson's equation

The FFT is one of the truly great computational developments of this [20th] century. It has changed the face of science and engineering so much that it is not an exaggeration to say that life as we know it would be very different without the FFT. *-Charles van Loan*





Polynomials: Coefficient Representation



- Polynomials: [coefficient representation]

$$A(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$$

$$B(x) = b_0 + b_1x + b_2x^2 + \cdots + b_{n-1}x^{n-1}$$

- Add: $O(n)$ arithmetic operations

$$A(x) + B(x) = (a_0 + b_0) + (a_1 + b_1)x + \cdots + (a_{n-1} + b_{n-1})x^{n-1}$$

- Evaluate: $O(n)$ using Horner's method

$$A(x) = a_0 + (x(a_1 + x(a_2 + \cdots + x(a_{n-2} + x(a_{n-1}))))$$

- Multiply (convolve): $O(n^2)$ using brute force

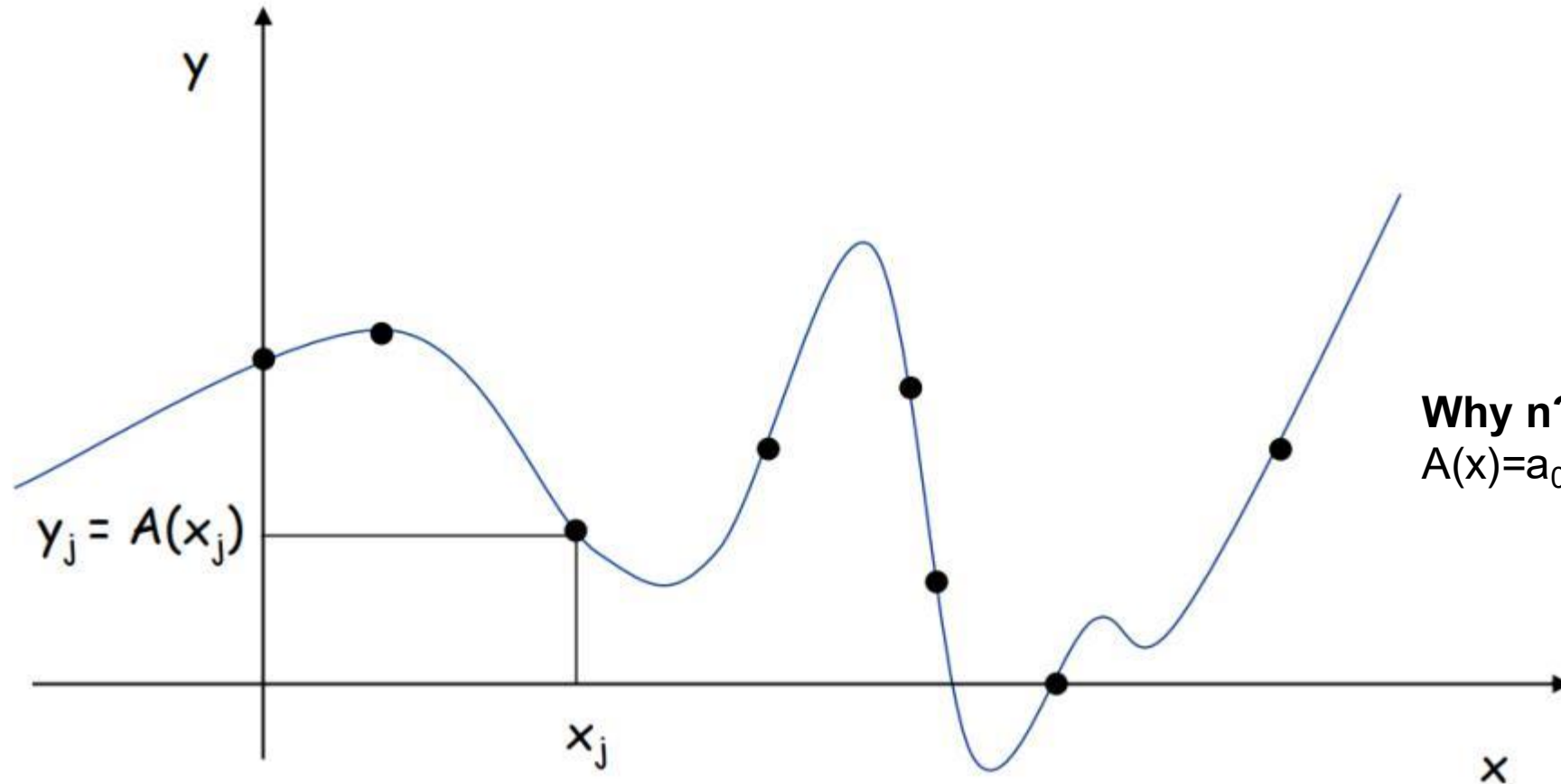
$$A(x) \times B(x) = \sum_{i=0}^{2n-2} c_i x^i, \text{ where } c_i = \sum_{j=0}^i a_j b_{i-j}$$





Polynomials: Point-Value Representation

- A degree **n-1** polynomial $A(x)$ is uniquely specified by its evaluation at **n** distinct values of x .



Why n?
 $A(x)=a_0+a_1x+a_2x^2$





Polynomials: Point-Value Representation



- Polynomial: [point-value representation]

$$A(x): (x_0, y_0), \dots, (x_{n-1}, y_{n-1}) \quad y_i = A(x_i)$$

$$B(x): (x_0, z_0), \dots, (x_{n-1}, z_{n-1})$$

- Add: $O(n)$ arithmetic operation

$$A(x) + B(x): (x_0, y_0 + z_0), \dots, (x_{n-1}, y_{n-1} + z_{n-1})$$

- Multiply: $O(n)$, but need $2n-1$ points

$$A(x) \times B(x): (x_0, y_0 \times z_0), \dots, (x_{2n-1}, y_{2n-1} \times z_{2n-1})$$

- Evaluate: $O(n^2)$ using Lagrange's formula

$$A(x) = \sum_{k=0}^{n-1} y_k \frac{\prod_{j \neq k} (x - x_j)}{\prod_{j \neq k} (x_k - x_j)}$$

$$A(x) = \sum_{k=0}^{n-1} y_k L_k(x)$$

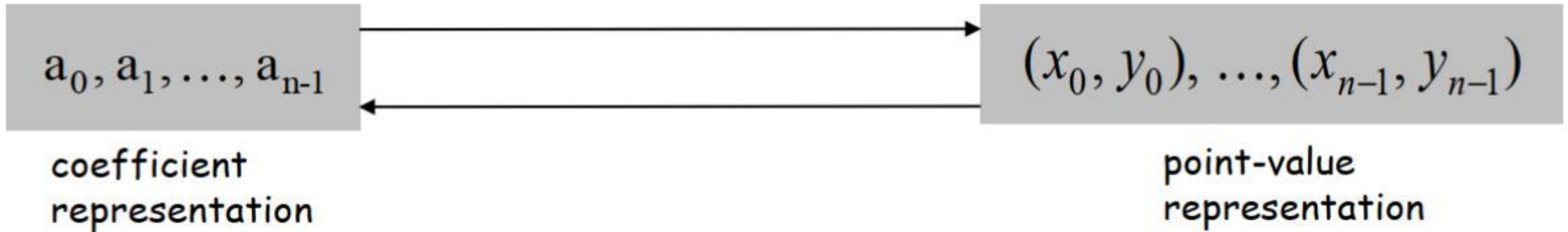


Converting Between Two Polynomial Representations

- Tradeoff. Fast evaluation or fast multiplication. We want both!

Representation	Multiply	Evaluate
Coefficient	$O(n^2)$	$O(n)$
Point-value	$O(n)$	$O(n^2)$

- Goal. Make all opt fast by efficiently converting between two representations





Converting Between Two Polynomial Representations: Brute Force



- **Coefficient $\rightarrow$ point-value.** Given a polynomial $a_0 + a_1x + \dots + a_{n-1}x^{n-1}$, evaluate it at n distinct points $x_0, \dots, x_{n-1}$.

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

Running time. $O(n^2)$ for matrix-vector multiplication or n times Horner's method





Converting Between Two Polynomial Representations: Brute Force

- **Point-value $\rightarrow$ Coefficient.** Given n distinct points $x_0, \dots, x_{n-1}$ and values $y_0, \dots, y_{n-1}$, find unique polynomial $a_0 + a_1x + \dots + a_{n-1}x^{n-1}$ that has given values at given points.

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ \vdots \\ y_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & x_0 & x_0^2 & \cdots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \cdots & x_1^{n-1} \\ 1 & x_2 & x_2^2 & \cdots & x_2^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \cdots & x_{n-1}^{n-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ \vdots \\ a_{n-1} \end{bmatrix}$$

$$y_0 = a_0 + a_1x_0 + a_2x_0^2 + \cdots + a_{n-1}x_0^{n-1} \quad Va = y \quad a = V^{-1}y$$

Running time. $O(n^3)$ for Gaussian elimination.





Coefficient to Point-Value Representation: Intuition

- **Coefficient to point-value.** Given a polynomial $a_0 + a_1x + \dots + a_{n-1}x^{n-1}$, evaluate it at n distinct points $x_0, \dots, x_{n-1}$.

We can choose which points!

- **Divide.** Break polynomial up into even and odd powers.

$$A(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4 + a_5x^5 + a_6x^6 + a_7x^7.$$

$$A_{\text{even}}(x) = a_0 + a_2x + a_4x^2 + a_6x^3. \quad a_0 + a_2x^2 + a_4x^4 + a_6x^6 \quad x^2, x^4, x^6 \quad (x^2)^1, (x^2)^2, (x^2)^3$$

$$A_{\text{odd}}(x) = a_1 + a_3x + a_5x^2 + a_7x^3. \quad a_1x + a_3x^3 + a_5x^5 + a_7x^7 \quad x(a_1 + a_3x^2 + a_5x^4 + a_7x^6)$$

$$A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2).$$

$$A(-x) = A_{\text{even}}(x^2) - x A_{\text{odd}}(x^2).$$

- **Intuition.** Choose two points to be ± 1

$$A(1) = A_{\text{even}}(1) + 1 A_{\text{odd}}(1).$$

$$A(-1) = A_{\text{even}}(1) - 1 A_{\text{odd}}(1).$$

Can evaluate polynomial of degree $\leq n$ at 2 points by evaluating two polynomials of degree $\leq \frac{1}{2}n$ at 1 points





Coefficient to Point-Value Representation: Intuition

- **Coefficient to point-value.** Given a polynomial $a_0 + a_1x + \dots + a_{n-1}x^{n-1}$, evaluate it at n distinct points $x_0, \dots, x_{n-1}$.

We can choose which points!

- **Divide.** Break polynomial up into even and odd powers.

$$A(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4 + a_5x^5 + a_6x^6 + a_7x^7.$$

$$A_{\text{even}}(x) = a_0 + a_2x + a_4x^2 + a_6x^3.$$

$$A_{\text{odd}}(x) = a_1 + a_3x + a_5x^2 + a_7x^3.$$

$$A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2).$$

$$A(-x) = A_{\text{even}}(x^2) - x A_{\text{odd}}(x^2).$$

- **Intuition.** Choose four complex points to be $\pm 1, \pm i$

$$A(1) = A_{\text{even}}(1) + 1 A_{\text{odd}}(1).$$

$$A(-1) = A_{\text{even}}(1) - 1 A_{\text{odd}}(1).$$

$$A(i) = A_{\text{even}}(-1) + i A_{\text{odd}}(-1).$$

$$A(-i) = A_{\text{even}}(-1) - i A_{\text{odd}}(-1).$$

Can evaluate polynomial of degree $\leq n$ at 4 points by evaluating two polynomials of degree $\leq \frac{1}{2}n$ at 2 points





Coefficient to Point-Value Representation: Intuition



- **Coefficient to point-value.** Given a polynomial $a_0 + a_1x + \dots + a_{n-1}x^{n-1}$, evaluate it at n distinct points $x_0, \dots, x_{n-1}$.

We can choose which points!

- **Divide.** Break polynomial up into even and odd powers.

$$A(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4 + a_5x^5 + a_6x^6 + a_7x^7.$$

$$A_{\text{even}}(x) = a_0 + a_2x + a_4x^2 + a_6x^3.$$

$$A_{\text{odd}}(x) = a_1 + a_3x + a_5x^2 + a_7x^3.$$

$$A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2).$$

$$A(-x) = A_{\text{even}}(x^2) - x A_{\text{odd}}(x^2).$$

- **Goal.** Choose n points s.t.
 - Can evaluate polynomial of degree $\leq n$ at n points by evaluating two polynomials of degree $\leq \frac{1}{2}n$ at $\frac{1}{2}n$ point
 - But also: can evaluate polynomial of degree $\leq \frac{1}{2}n$ at $\frac{1}{2}n$ points by evaluating two polynomials of degree $\leq \frac{1}{4}n$ at $\frac{1}{4}n$ point, and so on





Roots of Unity



- **Def.** An n^{th} **root of unity** is a complex number x such that $x^n = 1$
- **Fact.** The n^{th} roots of unity are : $w^0, w^1, \dots, w^{n-1}$ where $w = e^{2\pi i/n}$
- **Pf.** $(w^k)^n = (e^{2\pi i k/n})^n = (e^{2\pi i})^k = 1$ $e^{i\theta} = \cos \theta + i \sin \theta$ $e^{i\pi} = -1$

- **Fact.** The $\frac{1}{2}n^{\text{th}}$ roots of unity are: $v^0, v^1, \dots, v^{n/2-1}$ where $v = e^{4\pi i/n}$
- **Fact.** $w^2 = v$ and $(w^2)^k = v^k$

$$\omega = e^{2\pi i/n} \quad \omega^2 = (e^{2\pi i/n})^2 = e^{4\pi i/n}$$

$$\omega = e^{2\pi i/n} \quad \omega^2 = e^{4\pi i/n} \quad e^{4\pi i/n} = e^{2\pi i/(n/2)}$$

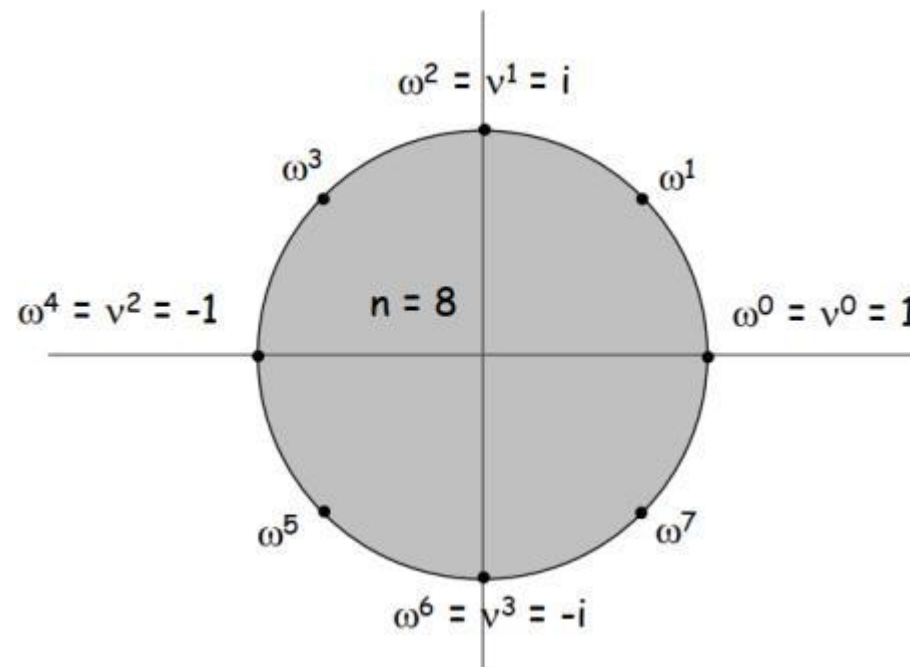
1		
ω	$1^2 = 1$	1
ω^2	ω^2	ω^2
ω^3	ω^4	ω^4
ω^4	ω^6	ω^6
ω^5	$\omega^8 = 1$	
ω^6		
ω^7		

$$\omega = e^{2\pi i/n} \quad \omega^k = e^{2\pi i k/n}$$

$$e^{2\pi i(k+n)/n} = e^{2\pi i k/n} \cdot e^{2\pi i}$$

$$\omega^{k+n} = \omega^k$$

$$\omega^k = \omega^{(k \bmod n)}$$





Discrete Fourier Transform

- **Coefficient to point-value.** Given a polynomial $a_0 + a_1x + \dots + a_{n-1}x^{n-1}$, evaluate it at n distinct points $x_0, \dots, x_{n-1}$
- **Key idea:** choose $x_k = w^k$ where w is principal n^{th} root of unity

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^1 & \omega^2 & \omega^3 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \omega^6 & \dots & \omega^{2(n-1)} \\ 1 & \omega^3 & \omega^6 & \omega^9 & \dots & \omega^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \omega^{3(n-1)} & \dots & \omega^{(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{bmatrix}$$



$$y_k = a_0 + a_1x_k + a_2x_k^2 + \dots + a_{n-1}x_k^{n-1}$$
$$y_k = a_0 + a_1\omega^k + a_2\omega^{2k} + \dots + a_{n-1}\omega^{(n-1)k}$$

Discrete Fourier Transform

Fourier matrix F_n

DFT=Fourier matrix $\times$ coefficient vector





Fast Fourier Transform



- **Goal.** Evaluate a degree $n-1$ polynomial $A(x) = a_0 + \dots + a_{n-1}x^{n-1}$ at its n^{th} roots of unity: $w^0, w^1, \dots, w^{n-1}$
- **Divide.** Break polynomial up into even and odd powers

$$A_{\text{even}}(x) = a_0 + a_2x + a_4x^2 + \dots + a_{n/2-2}x^{(n-1)/2}.$$

$$A_{\text{odd}}(x) = a_1 + a_3x + a_5x^2 + \dots + a_{n/2-1}x^{(n-1)/2}.$$

$$A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2).$$

- **Conquer.** Evaluate degree $A_{\text{even}}(x)$ and $A_{\text{odd}}(x)$ at the $\frac{1}{2}n^{\text{th}}$ roots of unity: $v^0, v^1, \dots, v^{n/2-1}$
- **Combine**

$$A(\omega^k) = A_{\text{even}}(v^k) + \omega^k A_{\text{odd}}(v^k), \quad 0 \leq k < n/2$$

$$A(\omega^{k+n/2}) = A_{\text{even}}(v^k) - \omega^k A_{\text{odd}}(v^k), \quad 0 \leq k < n/2$$

$$\begin{array}{c} \uparrow \\ \omega^{k+n/2} = -\omega^k \end{array} \quad \omega^{n/2} = e^{\pi i} = -1 \quad \omega^{k+n/2} = \omega^k(-1)$$

$$v^k = (\omega^k)^2 = (\omega^{k+n/2})^2$$





Fast Fourier Transform

- **Goal.** Evaluate a degree $n-1$ polynomial $A(x) = a_0 + \dots + a_{n-1}x^{n-1}$ at its n^{th} roots of unity: $w^0, w^1, \dots, w^{n-1}$

$$A_{\text{even}}(x) = a_0 + a_2x + a_4x^2 + \dots + a_{n/2-2}x^{(n-1)/2}.$$

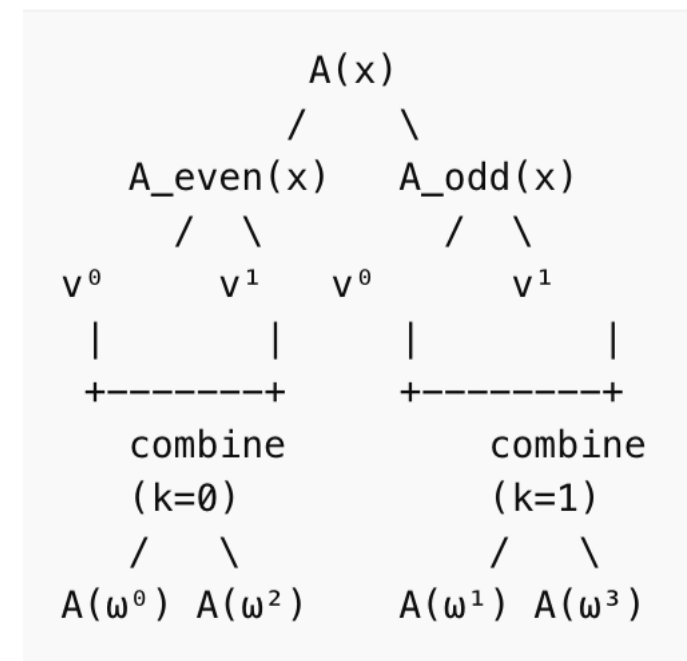
$$A_{\text{odd}}(x) = a_1 + a_3x + a_5x^2 + \dots + a_{n/2-1}x^{(n-1)/2}.$$

$$A(x) = A_{\text{even}}(x^2) + x A_{\text{odd}}(x^2).$$

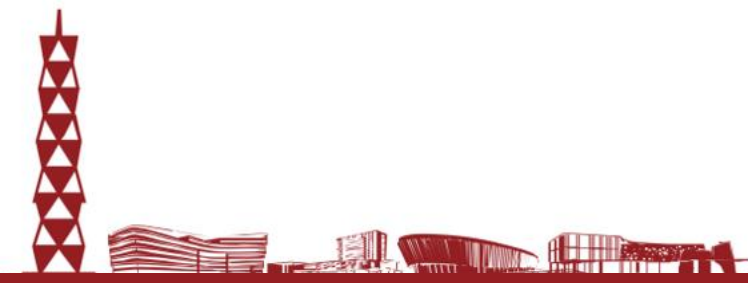
$$A(\omega^k) = A_{\text{even}}(v^k) + \omega^k A_{\text{odd}}(v^k), \quad 0 \leq k < n/2$$
$$A(\omega^{k+n/2}) = A_{\text{even}}(v^k) - \omega^k A_{\text{odd}}(v^k), \quad 0 \leq k < n/2$$

$$\uparrow$$
$$\omega^{k+n/2} = -\omega^k$$

$$v^k = (\omega^k)^2 = (\omega^{k+n/2})^2$$



$$A(\omega^k) = A_{\text{even}}(v^k) + \omega^k A_{\text{odd}}(v^k)$$
$$A(\omega^{k+2}) = A_{\text{even}}(v^k) - \omega^k A_{\text{odd}}(v^k)$$





FFT Algorithm



```
FFT(n, a0, a1, ..., an-1) {  
    if (n == 1) return a0  
  
    (e0, e1, ..., en/2-1) ← FFT(n/2, a0, a2, a4, ..., an-2)  
    (d0, d1, ..., dn/2-1) ← FFT(n/2, a1, a3, a5, ..., an-1)  
  
    for k = 0 to n/2 - 1 {  
        ωk ← e2πik/n  
        Yk ← ek + ωk dk  
        Yk+n/2 ← ek - ωk dk  
    }  
  
    return (Y0, Y1, ..., Yn-1)  
}
```



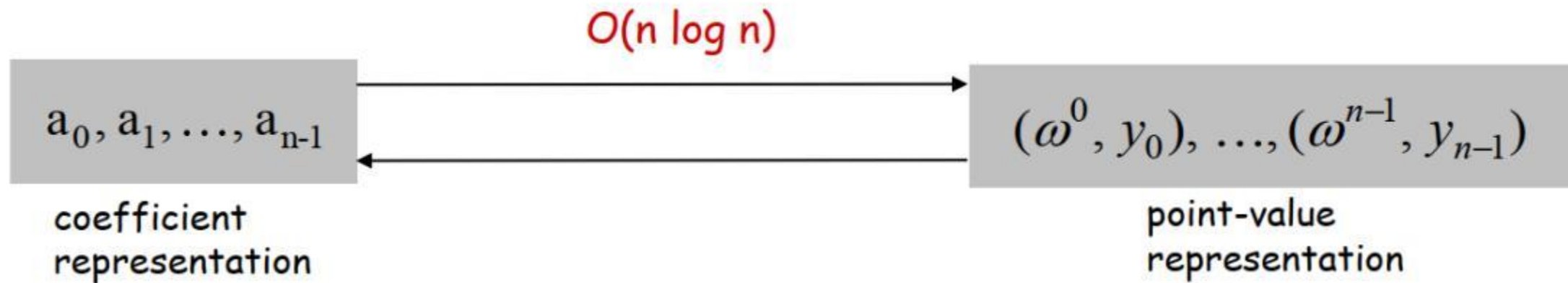


FFT Summary

- Theorem. FFT algorithm evaluates a degree $n-1$ polynomial at each of the n^{th} roots of unity in $O(n \log n)$ steps

↑
assumes n is a power of 2

- Pf. $T(2n) = 2T(n) + O(n) \rightarrow T(n) = O(n \log n)$





Point-Value to Coefficient Representation: Inverse DFT

- **Goal.** Given the values $y_0, \dots, y_{n-1}$ of a degree $n-1$ polynomial at the n points $w^0, w^1, \dots, w^{n-1}$, find unique polynomial $a_0 + a_1x + \dots + a_{n-1}x^{n-1}$ that has given values at given points.

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^1 & \omega^2 & \omega^3 & \dots & \omega^{n-1} \\ 1 & \omega^2 & \omega^4 & \omega^6 & \dots & \omega^{2(n-1)} \\ 1 & \omega^3 & \omega^6 & \omega^9 & \dots & \omega^{3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{n-1} & \omega^{2(n-1)} & \omega^{3(n-1)} & \dots & \omega^{(n-1)(n-1)} \end{bmatrix}^{-1} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{bmatrix}$$

↑
Inverse DFT

↑
Fourier matrix inverse $(F_n)^{-1}$





Inverse FFT



- **Claim.** Inverse of Fourier matrix is given by following formula.

$$G_n = \frac{1}{n} \begin{bmatrix} 1 & 1 & 1 & 1 & \dots & 1 \\ 1 & \omega^{-1} & \omega^{-2} & \omega^{-3} & \dots & \omega^{-(n-1)} \\ 1 & \omega^{-2} & \omega^{-4} & \omega^{-6} & \dots & \omega^{-2(n-1)} \\ 1 & \omega^{-3} & \omega^{-6} & \omega^{-9} & \dots & \omega^{-3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{-(n-1)} & \omega^{-2(n-1)} & \omega^{-3(n-1)} & \dots & \omega^{-(n-1)(n-1)} \end{bmatrix}$$





Inverse FFT: Proof of Correctness



- **Claim.** F_n and G_n are inverses.

- **Pf.**

$$(F_n G_n)_{kk'} = \frac{1}{n} \sum_{j=0}^{n-1} \omega^{kj} \omega^{-jk'} = \frac{1}{n} \sum_{j=0}^{n-1} \omega^{(k-k')j} = \begin{cases} 1 & \text{if } k = k' \\ 0 & \text{otherwise} \end{cases}$$

↑
summation lemma

- **Summation lemma.** Let ω be a principal n^{th} root of unity Then

$$\sum_{j=0}^{n-1} \omega^{kj} = \begin{cases} n & \text{if } k \equiv 0 \pmod{n} \\ 0 & \text{otherwise} \end{cases}$$

- **Pf.**

If k is a multiple of n then $\omega^k = 1 \Rightarrow$ sums to n .

Else: $\omega^k \neq 1$

- $x^n - 1 = (x - 1)(1 + x + x^2 + \dots + x^{n-1})$

- Let $x = \omega^k$, we have $0 = \omega^{kn} - 1 = (\omega^k - 1)(1 + \omega^k + \omega^{k(2)} + \dots + \omega^{k(n-1)})$

- Therefore, $1 + \omega^k + \omega^{k(2)} + \dots + \omega^{k(n-1)} = 0 \Rightarrow$ sums to 0. ▀





Inverse FFT: Proof of Correctness



- **Claim.** F_n and G_n are inverses.
- **Pf.**

$$(F_n G_n)_{kk'} = \frac{1}{n} \sum_{j=0}^{n-1} \omega^{kj} \omega^{-jk'} = \frac{1}{n} \sum_{j=0}^{n-1} \omega^{(k-k')j} = \begin{cases} 1 & \text{if } k = k' \\ 0 & \text{otherwise} \end{cases}$$

↑
summation lemma

$$k - k' = 0$$

$$\omega^{(k-k')j} = \omega^0 = 1$$

$$S = 1 + 1 + 1 + \dots + 1$$

$$S = n$$

$$m = k - k'$$

$$S = \sum_{j=0}^{n-1} \omega^{mj}$$

$$S = 1 + \omega^m + \omega^{2m} + \dots + \omega^{(n-1)m}$$

$$r = \omega^m \quad S = \frac{r^n - 1}{r - 1}$$

$$S = \frac{1 - 1}{\omega^m - 1} = 0 \quad m \neq 0$$

$$\begin{array}{ccc} & \omega^2 & \\ \omega^3 & & \omega^1 \\ \omega^4 & & 1 \\ & \omega^5 & \end{array}$$





Inverse FFT: Proof of Correctness



- **Claim.** F_n and G_n are inverses.

- **Pf.**

$$(F_n G_n)_{kk'} = \frac{1}{n} \sum_{j=0}^{n-1} \omega^{kj} \omega^{-jk'} = \frac{1}{n} \sum_{j=0}^{n-1} \omega^{(k-k')j} = \begin{cases} 1 & \text{if } k = k' \\ 0 & \text{otherwise} \end{cases}$$

↑
summation lemma

- **Summation lemma.** Let ω be a principal n^{th} root of unity Then

$$\sum_{j=0}^{n-1} \omega^{kj} = \begin{cases} n & \text{if } k \equiv 0 \pmod{n} \\ 0 & \text{otherwise} \end{cases} \quad \omega = e^{2\pi i/n} \quad \omega^n = 1 \quad \omega^{n+m} = \omega^m$$

- **Pf.**

If k is a multiple of n then $\omega^k = 1 \Rightarrow$ sums to n .

Else: $\omega^k \neq 1$

$$- x^n - 1 = (x - 1)(1 + x + x^2 + \dots + x^{n-1})$$

$$- \text{Let } x = \omega^k, \text{ we have } 0 = \omega^{kn} - 1 = (\omega^k - 1)(1 + \omega^k + \omega^{k(2)} + \dots + \omega^{k(n-1)})$$

$$- \text{Therefore, } 1 + \omega^k + \omega^{k(2)} + \dots + \omega^{k(n-1)} = 0 \Rightarrow \text{sums to } 0. \blacksquare$$





Point-Value to Coefficient Representation: Inverse DFT

$$\begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_{n-1} \end{bmatrix} = \frac{1}{n} \begin{bmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega^{-1} & \omega^{-2} & \omega^{-3} & \cdots & \omega^{-(n-1)} \\ 1 & \omega^{-2} & \omega^{-4} & \omega^{-6} & \cdots & \omega^{-2(n-1)} \\ 1 & \omega^{-3} & \omega^{-6} & \omega^{-9} & \cdots & \omega^{-3(n-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega^{-(n-1)} & \omega^{-2(n-1)} & \omega^{-3(n-1)} & \cdots & \omega^{-(n-1)(n-1)} \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_{n-1} \end{bmatrix}$$

- **Observation.** Almost the same form as DFT.
- **Consequence.** To compute inverse FFT, apply same algorithm but use $w^{-1} = e^{-2\pi i/n}$ as principal n^{th} root of unity (and divide by n)

$$a_j = \frac{1}{n} \sum_{k=0}^{n-1} y_k e^{-2\pi i j k / n}$$





Inverse FFT: Algorithm



```
IFFT(n, a0, a1, ..., an-1) {  
    if (n == 1) return a0  
  
    (e0, e1, ..., en/2-1) ← IFFT(n/2, a0, a2, a4, ..., an-2)  
    (d0, d1, ..., dn/2-1) ← IFFT(n/2, a1, a3, a5, ..., an-1)  
  
    for k = 0 to n/2 - 1 {  
        ωk ← e-2πik/n  
        yk ← (ek + ωk dk)  
        yk+n/2 ← (ek - ωk dk)  
    }  
  
    return (y0, y1, ..., yn-1)  
}
```

y₀, y₁, ..., y_(n-1)

$$a_j = \frac{1}{n} \sum y_k \omega^{-jk}$$

$$y_k = \sum a_j \omega^{kj}$$

Note. Need to divide the final result by n



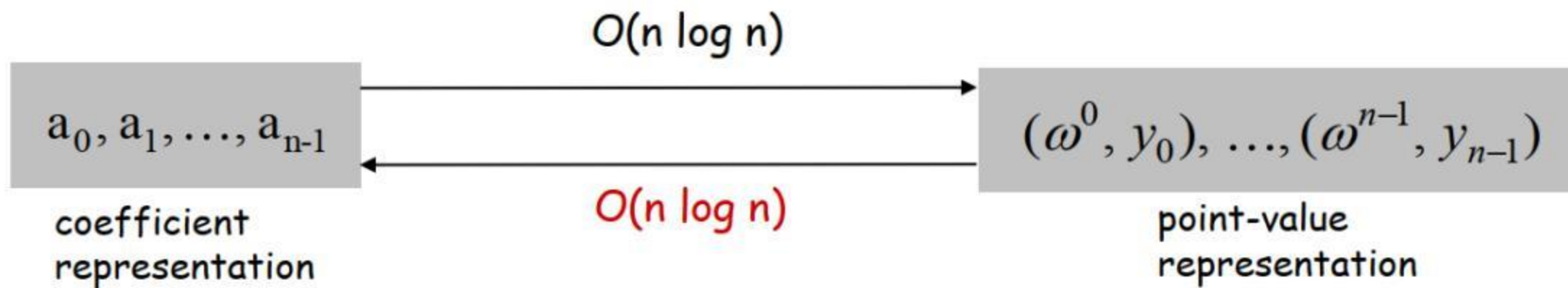


Inverse FFT Summary



- **Theorem.** Inverse FFT algorithm interpolates a degree $n-1$ polynomial given values at each of the n^{th} roots of unity in $O(n \log n)$ steps

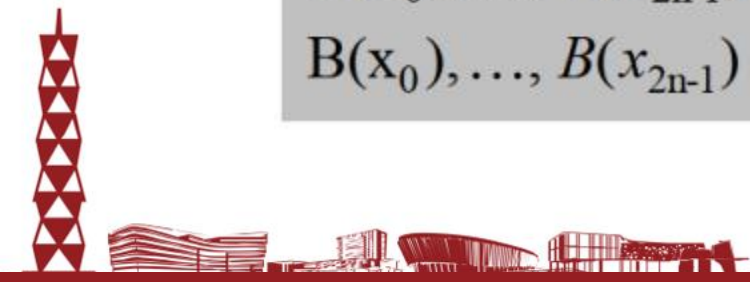
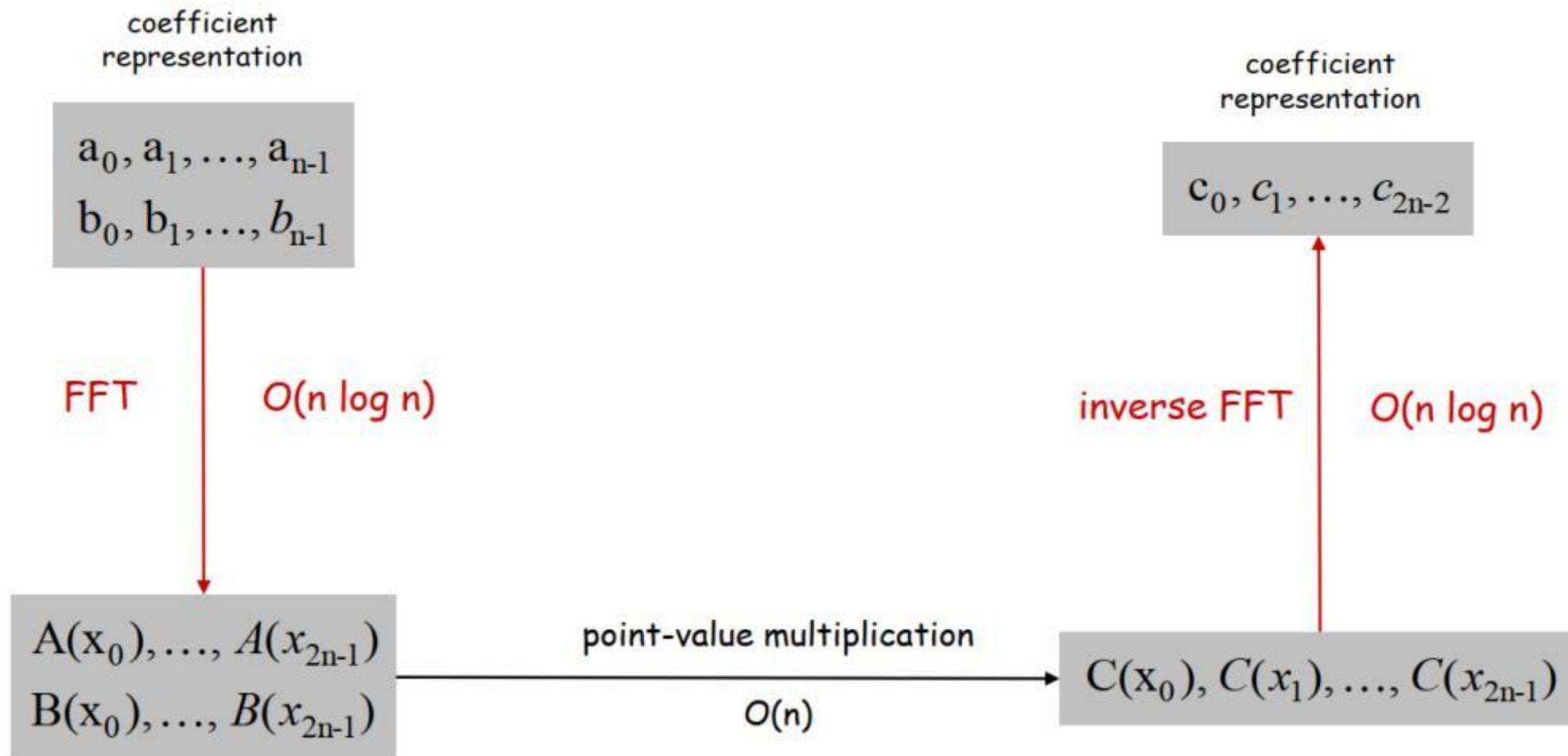
↑
assumes n is a power of 2





Polynomial Multiplication

- **Theorem.** Can multiply two degree $n-1$ polynomials in $O(n \log n)$ steps





Divide-and-Conquer Summary



- **Basic idea**

- Break up problem into several parts
- Solve each part recursively
- Combine solutions to sub-problems into overall solution

- **Algorithms**

- Mergesort
 - Divide a sequence into two of same size
- Closest Pair of Points
 - Vertically divide the space
- Integer Multiplication
 - Divide each n -digit integer into two $\frac{1}{2}n$ -digit integers
- Matrix Multiplication
 - Divide each n -by- n matrix into four $\frac{1}{2}n$ -by- $\frac{1}{2}n$ blocks
- Fast Fourier Transform
 - Divide a polynomial into two with even and odd powers

